7.Puntatori

è una variabile che come valore ha l'indirizzo di memoria di un altra variabile , questa oltre a rappresentare l'indirizzo ne rappresenta anche il tipo dell'oggetto.
Se un oggetto o una funzione abbia il tipo `T` , anche il puntatore dovrà avere il tipo `t` , per esempio se la variabile `var` è un tipo *float* , l'espressione `&var` , che sarebbe il valore dell'indirizzo della variabile float , sarà un puntatore di tipo *float* .

```
type * [type-qualifier-list] name [= initializer];
```

L'operatore indirzzo & restituisce l'indirizzo del suo operatore , se l'operatore x ha il tipo T , la sua espressione `&x` sarà un puntatore di tipo T
Per ottenere l'indirizzo di un oggetto o di una funzione sarà necessario inizializzare il puntatore in questo modo :

```
float x , *ptr;
ptr = &x;
```

Quando invece abbiamo un puntatore che vuole accedere all'oggetto in cui fa riferimento , si dovrà usare l'operatore `*` .
Se `ptr` è un puntatore , `*prt` rappresenterà l'oggetto o la funzione che `ptr` punta , quando questa è un puntatore di un oggetto , `*ptr` sarà un ***lvalue*** , e questo si potrà esser usato come operando sinistro di un operatore di assegnazione:

```
float x, *ptr = &x;

*ptr = 1.7

++(*ptr);
```

Print Pointers

Spesso è anche utile stampare in output per verificare l'indirizzo è svolgere vari debug, e la funzione ***printf()*** prevede un formato specifico per stampare i puntatore;

- %p
  E la dimensione di un puntatore in memoria ,è la stessa indipendentemente dal tipo di oggetto indirizzato

Null Pointers

Un puntatore costante nullo è un intero che ha come valore 0

Void Pointers

Un puntatore void , o un void puntatore , è un puntatore che ha come tipo il `void *` , e questo tipo viene usato come tipo di puntatore universale, questo puo rappresentare l'indirizzo di memoria di qualsiasi oggetto , MA NON IL SUO TIPO, e per accedere all'indirizzo di memoria di un oggetto dovremmo convertire il tipo del puntatore void a quello dell'oggetto :

```c
void *pA = NULL;
int p = 10;
pA = &p;
printf("%d", *((int*) pA));
```

Puntatori a Puntatori

Un puntatore è comunque un oggetto in memoria, questo significa, che un puntatore può puntarlo, e per dichiarare un puntatore che punta a un puntatore , dovremmo usare due `**`

```c
char c = 'A', *cPtr = &c, **cPtrPtr = &cPtr;
```

Leggere e modificare

Se `ptr` è un puntatore , `*ptr` rappresenta l'oggetto che ptr punta , il tipo del puntatore determina il tipo dell'oggetto che sta assumendo per esser allocato in memoria

Aritmetica dei puntatore

quando si svolge l'aritmetica dei puntatore , il compilatore adatta le operazioni alle dimensioni degli oggetti a cui si riferisce il tipo del puntatore
Ed è possibile svolgere le seguenti operazione ai puntatori:

- aggiunta o sottrazione di un int
- sottrazione di un puntatore a un altro
- confronto tra 2 pointer

L VALUES e Puntatori

Gli operatori che generano un *lvalue* includono l'operatore di sottoscrizione (`[]`) e l'operatore di indirezione (`*`).

- array[1] = Sì; un elemento dell'array è un oggetto con una posizione in memoria.
- &array[1] = No; la posizione dell'oggetto non è un oggetto con una posizione
- ptr = si ; il puntatore è un oggetto con una posizione
-
    - ptr = si ; cosa il puntatore punta a cosa è comunque un oggetto con posizione
- ptr + 1 = no ; l'addizione assegna una nuova posizione ma non un oggetto
-
    - ptr + 1 = no ; addizione segna un nuovo valore aritmetico ma non l'oggetto

Const pointers e puntatori a costanti

È possibile definire un puntatore come costante, e quando lo dichiari lo dovrai inizializzare perchè non potrai più modificarlo successivamente
Possiamo modificare un puntatore che punta a un oggetto che ha una tipo di costante qualificata, ma allo stesso tempo potremmo usare un puntatore solamente per leggere un determinato oggetto

Perche i puntatori?

un puntatore è un riferimento a un oggetto o a una funzione , e hanno molti utilizzi:

- definire funzioni "call-by-reference"
- implementare strutture di dati dinamiche

Call by reference

è un modo semplice di lavorare nella stessa memoria ma da diverse funzioni.

Function Swap

Per modificare un puntatore , definita nel main , dentro un altra funzione, non possiamo passare il valore , ma il suo indirizzo ***&var*** , la funzione ricevente deve avere un puntatore ptr e modificare il valore usando la dereferenziazione (ptr = *nuovo_valore*)

```
void swap(int *p,int *q)
{
int tmp = *p; //salvo valore puntato da p
*p = *q; //metto valore q dentro p
*q = tmp; //metto valore tmp dentro q
}
```

Quando non usare le var globali

Prima di tutto dobbiamo dire , che dobbiamo usare la var globali il meno possibile perche:

- usare una var locale ti permette di focussarti con molta attenzione su piu piccole parti del programma . Le globali ti forzano di tenere traccia di come sono usare in tutto il sistema
- Non è valido creare due var globali con lo stesso nome

Quando invece usarla?

Se l'intero programma è costruito attorno a dei dati centrali e l'accesso a questi dati è necessario in quasi tutte le funzioni, specialmente se si tratta di una grande struttura di dati, allora ha senso rendere tali dati globali.